河南财经政法大学

计算机与信息工程学院

《高级程序设计实践》

集中实践报告

评　　分	
姓　　名	石梓琛
学　　号	202434072129
专业名称	软件工程（金融信息化方向）
教学班号	高级程序设计-0007
指导教师	满江涛

2026 年 6 月 18 日

一、软件功能

本软件是一个运行在 Windows 桌面环境下的屏幕使用时间管理工具，采用 JavaFX 构建图形界面，使用 SQLite 作为本地数据库，通过 JNA 调用 Windows 原生 API 获取前台窗口信息。软件以单进程桌面应用形式独立运行，无需服务器端。

主要功能包括：

（1）监控软件管理：用户可以刷新当前系统正在运行的进程列表，通过勾选方式将指定进程添加到监控列表，也可以从监控列表中移除应用。被移除的应用采用软删除方式保留历史数据。

（2）前台应用监控：程序在后台以固定间隔定时检测当前前台窗口所属进程。若该进程在监控列表中，则自动开始记录使用会话；若应用切换或窗口失焦，则自动结束当前会话并记录持续时长。

（3）今日使用统计：以表格形式展示当天各被监控应用的累计使用时长（含进行中的会话），支持按时长排序查看。

（4）历史记录：以表格形式展示近 31 天的历史使用记录，按应用和日期分组呈现，支持筛选查看。

（5）系统托盘与后台运行：关闭主窗口后程序继续在后台运行并自动最小化到系统托盘；托盘图标右键菜单可重新显示主窗口或退出程序；双击托盘图标也可恢复窗口。

二、软件架构

本软件由一个可独立运行的 Windows 桌面程序组成，不包含独立的服务器端和客户端，所有功能运行在同一个 Java 进程中。

程序内部按职责划分为 5 层：

• 界面层（JavaFX 页面 + 控制器）：App.java / MainController / MonitorListController / StatisticsController / HistoryController，以及对应的 FXML 布局文件。

• 服务层（后台监控）：ForegroundMonitorService，使用 ScheduledExecutorService 后台守护线程，定时轮询前台窗口状态。

• 数据访问层（DAO）：MonitoredAppDao / UsageRecordDao，负责 SQLite 数据库的增删改查。

• 工具层：DatabaseManager（连接管理）/ WindowsNativeUtil（Windows API 封装）/ IconUtil（图标加载）/ TimeFormatter（时间格式化）。

• 数据模型层：MonitoredApp / UsageRecord / ApplicationInfo 等 Java record/POJO 类。

数据库使用本地 SQLite（screentime.db），共 3 张表：monitored_apps（监控列表）、usage_records（使用会话）、daily_summary（日汇总）。

三、开发过程

本项目从 2026 年 6 月 6 日启动，至 6 月 28 日完成，历时约 3 周，通过 Git 版本控制共完成 38 次提交，经历 14 个主要开发任务。

第一阶段（第 1~2 周）：完成项目基础设施搭建（Maven 项目、依赖配置、Git 仓库）、数据库设计与初始化、主界面框架与页面导航、系统托盘功能（含中文显示 Bug 修复）、Windows 原生 API 集成、前台应用监控服务的核心逻辑，以及使用记录数据访问层（UsageRecordDao，854 行）的全量实现。

第二阶段（第 3 周）：完成各功能页面的实现（监控管理、今日统计、历史记录）、功能集成联调、Bug 修复与代码优化，以及前台监测可靠性增强（时钟回拨检测、轮询停滞检测、空闲超时检测）和 jpackage 打包部署。

四、用到哪些技术

（1）Java 21：项目主要开发语言。使用 Java 21 LTS，利用 record 类型、文本块、密封类等特性。
（2）JavaFX 21：构建桌面图形界面。使用 Stage/Scene 窗口管理、FXML 声明式布局、TableView、ListView、CheckBox、Spinner、Task 等组件；使用 PropertyValueFactory 进行数据绑定；通过 Platform.runLater() 实现线程间 UI 更新。

（3）FXML + CSS：声明式 UI 布局，使用 FXMLLoader 动态加载页面，通过 CSS 样式表统一界面风格。
（4）Maven：项目构建与依赖管理。使用 maven-compiler-plugin（release=21）和 javafx-maven-plugin；通过 maven-assembly-plugin 打包 Fat JAR。
（5）SQLite + JDBC：本地关系型数据库存储。使用 SQLite JDBC 3.49.1.0，通过 PreparedStatement 防 SQL 注入，使用事务保证 ACID，精细管理 Connection/Statement/ResultSet 资源。
（6）DAO 设计模式：数据访问层封装。MonitoredAppDao、UsageRecordDao 分别负责不同表的 CRUD，与业务逻辑解耦。
（7）JNA（Java Native Access）：通过 JNA Platform 5.14.0 调用 Windows 原生 API：User32.GetForegroundWindow()、GetWindowThreadProcessId()；Kernel32.OpenProcess()、QueryFullProcessImageNameW()，获取前台窗口所属进程的名称与路径。
（8）AWT SystemTray：系统托盘功能，使用 java.awt.SystemTray + TrayIcon + PopupMenu 实现托盘图标和右键菜单，与 JavaFX 通过 Platform.runLater() 协作。
（9）并发编程：使用 ScheduledExecutorService 创建后台定时任务线程；使用 JavaFX Task 执行耗时操作（进程枚举）以避免 UI 卡顿；通过 volatile 保证可见性。
（10）JUnit 5：单元测试框架。测试重点覆盖 UsageRecordDao 的旧会话处理、异常退出恢复、跨天拆分、历史统计不重复计数等，共 10 个测试全部通过。
（11）jpackage 打包：将应用打包为原生 Windows EXE/MSI 安装包，通过 Main.java 包装器解决 jpackage 与 JavaFX 模块查找的兼容问题。
（12）Git 版本控制：使用 Git 进行版本管理，共 38 次提交，完整记录从初始化到发布的全开发过程。

五、任务记录

（本实践共有 14 个任务，按功能模块分组展示如下，代码截图请在各任务后附上）

任务 1：项目初始化与环境搭建
目的：创建 Maven 项目结构，配置 Java 21 编译环境和核心依赖，初始化 Git 仓库。
内容：
创建 pom.xml，配置 JavaFX 21.0.11、SQLite JDBC 3.49.1.0、JNA Platform 5.14.0、JUnit 5.10.3 等核心依赖。
设置 maven-compiler-plugin（release=21）和 javafx-maven-plugin 插件。
创建标准 Maven 目录结构（src/main/java、src/main/resources 等）。
初始化 Git 仓库并配置 .gitignore。

任务 2：数据库设计与初始化

目的：设计并实现本地 SQLite 数据库结构，编写建表脚本和初始化代码。
内容：
设计 3 张数据表：monitored_apps（被监控应用列表，含软删除标记）、usage_records（使用会话记录，含开始/结束时间和持续秒数）、daily_summary（按天汇总）。
编写 schema.sql 建表脚本，定义表结构、索引和约束。
实现 DatabaseManager.java 单例类，管理 SQLite JDBC 连接，程序启动时自动执行建表脚本。

任务 3：主界面框架与页面导航
目的：搭建 JavaFX 主窗口框架，实现页面切换导航机制。
内容：
实现 App.java 作为 JavaFX Application 入口类，加载 main.fxml 主界面布局（左侧功能导航栏 + 右侧 StackPane 内容区）。
实现 MainController.java，处理导航按钮点击，通过 FXMLLoader 动态加载并切换不同 FXML 页面。
配置窗口标题、默认大小（900×600）、最小尺寸约束，引入 JavaFX CSS 样式表统一视觉风格。

任务 4：系统托盘功能
目的：实现关闭窗口时自动最小化到系统托盘，并修复托盘右键菜单中文显示异常。
内容：
使用 AWT SystemTray 和 TrayIcon 创建任务栏托盘图标，实现 PopupMenu 右键菜单（显示主窗口 / 退出程序）。
处理窗口关闭事件：隐藏窗口并最小化到系统托盘而非直接退出，双击托盘图标可恢复窗口。
修复托盘菜单中文显示为方块的 Bug：根因为 JDK 21 JEP 400 将 file.encoding 默认改为 UTF-8 后 fontconfig.properties 缺少中文字体序列，通过自定义 fontconfig.properties 文件解决。

任务 5：Windows 原生 API 集成
目的：通过 JNA 调用 Windows 原生 API，获取前台窗口的进程信息。
内容：
实现 WindowsNativeUtil.java，封装 User32.GetForegroundWindow()、GetWindowThreadProcessId()、Kernel32.QueryFullProcessImageNameW() 等 Windows API 调用。
实现 ApplicationInfo 记录类（进程名、路径、窗口标题等），作为前台窗口信息的数据载体。
添加以 PID 为 Key 的本地缓存，减少重复 Windows API 调用开销。

任务 6：前台应用监控服务
目的：实现后台定时检测前台窗口并记录使用时长的核心监控服务。
内容：

实现 ForegroundMonitorService.java 单例服务，使用 ScheduledExecutorService 按固定间隔（默认 15 秒）定时执行 poll() 轮询。
poll() 逻辑：获取前台进程 → 与上次对比 → 若切换则结束旧会话并开始新会话；若未切换则更新心跳。
支持跨午夜会话拆分记录，程序启动时通过 recoverOpenSessions() 恢复异常中断的会话。

任务 7：使用记录数据访问层
目的：实现使用记录的完整 CRUD 操作和复杂数据查询（UsageRecordDao.java，854 行）。
内容：
核心方法：startSession()（事务写入）、endSession()（计算持续秒数，支持跨午夜拆分）、heartbeatSession()（更新心跳）、recoverOpenSessions()（启动恢复）。
查询方法：getTodayUsage()（今日统计）、getRecentUsageHistory()（近 31 天历史）、getDailySummaries()（日汇总）等。
使用 PreparedStatement 防 SQL 注入，合理使用事务保证 ACID，精细管理数据库连接资源释放。

任务 8：监控应用管理页面
目的：实现用户选择和管理被监控应用的完整界面。
内容：
实现 MonitorListController.java 和 monitor-list.fxml，使用 ProcessHandle.allProcesses() 枚举当前系统进程，后台线程加载进程图标，避免阻塞 UI。
ListView + CheckBox 交互：用户勾选进程添加到监控列表，支持刷新进程列表和已监控应用的删除管理。
通过 MonitoredAppDao 实现软删除（is_active=0），保留历史使用记录关联。

任务 9：今日使用统计页面
目的：以表格形式展示当天各应用的使用时长统计数据。
内容：
实现 StatisticsController.java 和 statistics.fxml，使用 JavaFX TableView 展示应用图标、应用名称和今日累计使用时长。
调用 UsageRecordDao.getTodayUsage() 同时汇总已完成和进行中会话，支持按时长排序。
格式化时长显示为「X 小时 Y 分钟」可读格式，通过 PropertyValueFactory 绑定数据模型。

任务 10：历史记录页面
目的：以详细表格形式展示历史使用记录，方便用户回溯。
内容：
实现 HistoryController.java 和 history.fxml，使用 JavaFX TableView 按应用名称和日期分组展示近 31 天历史使用时长。

调用 UsageRecordDao.getRecentUsageHistory() 获取数据，结合已完成会话和进行中会话的实时时长。
支持按日期范围和应用名称筛选，格式化时间显示便于快速查看。

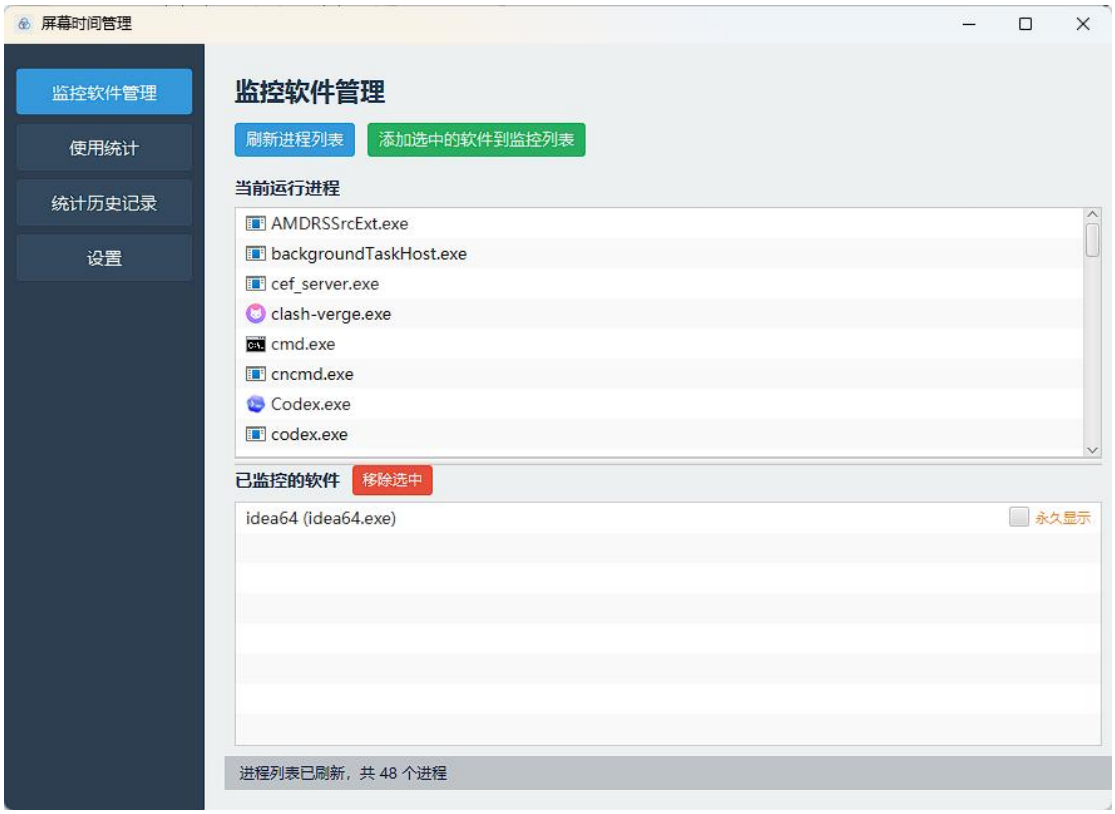
任务 11：功能集成与联调
目的：将所有功能模块进行集成联调，确保整体数据流畅通。
内容：
确保前台监控服务产生使用记录 → DAO 持久化 → 统计页面读取展示的完整数据流。
修复模块间接口不一致问题，统一数据对象和调用约定。
完成所有功能页面的完整交互流程测试，验证边界场景（空数据展示、应用切换频繁时的数据准确性）。

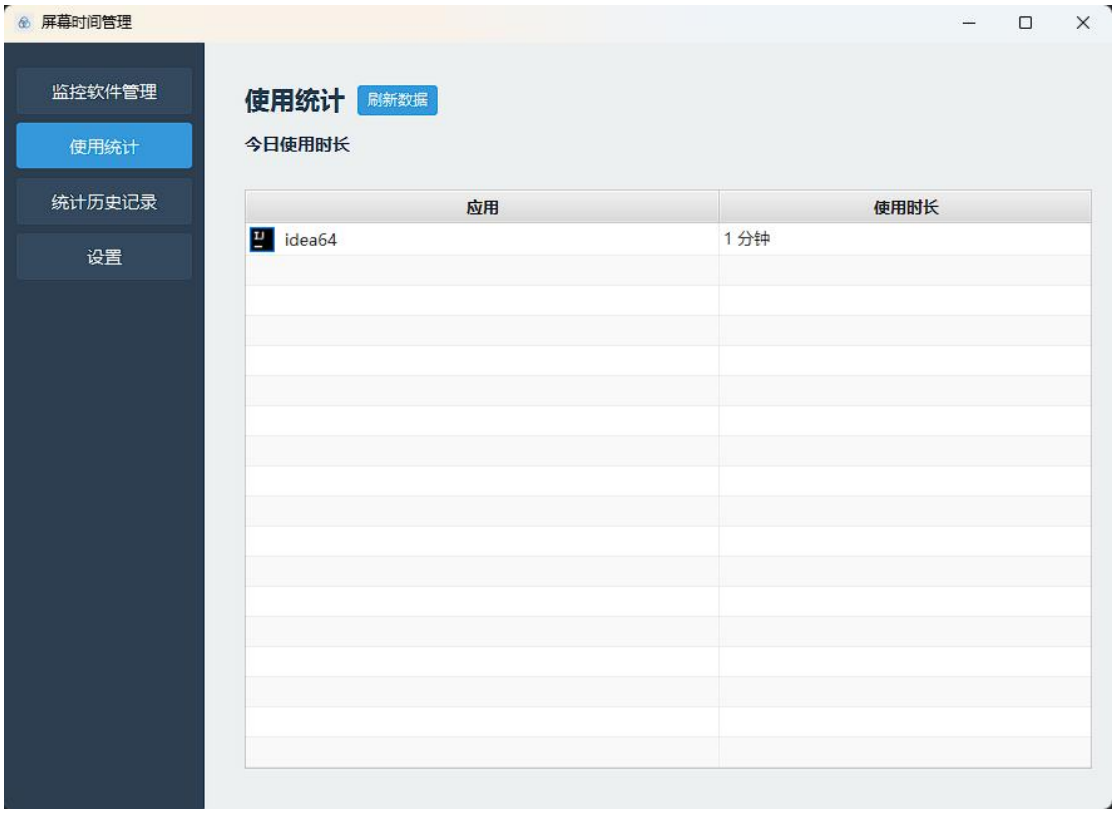
任务 12：Bug 修复与代码优化
目的：修复已知 Bug，优化代码质量和运行性能。
内容：
修复主要 Bug：系统托盘中文显示问题、数据库路径初始化问题、删除监控应用时的级联处理、空引用清理。
进行代码审查和重构：提取公共逻辑、改进异常处理（不再吞掉异常）、添加关键路径日志输出。
优化内存使用（缓存清理策略、资源及时释放）；标准化所有文件编码为 UTF-8，行尾格式统一为 LF。

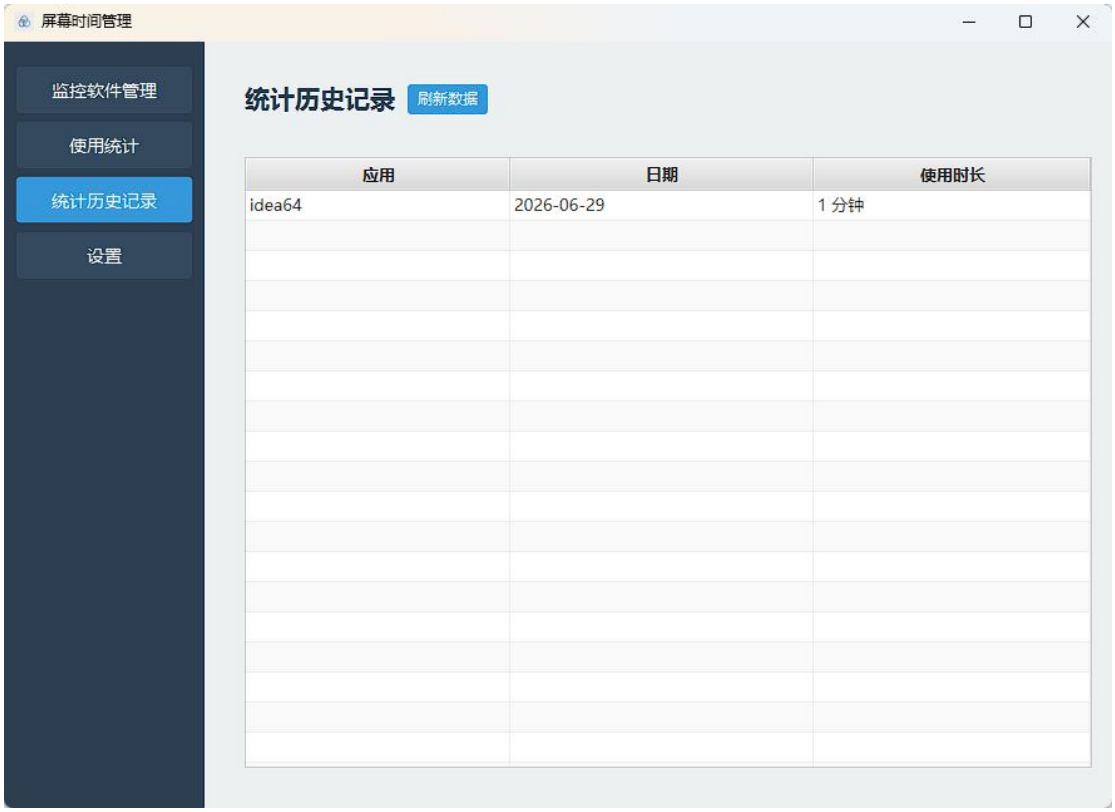
任务 13：前台监测可靠性修复与项目完善
目的：修复前台监测的深层次可靠性问题，完善项目文档。
内容：
在 ForegroundMonitorService 中添加三类安全检测：时钟回拨检测（回拨超 5 秒时安全关闭会话）、轮询停滞检测（通过 nanoTime 判断异常间隔）、空闲超时检测（安全关闭长时间无变化的会话）。
修复 UsageRecordDao.recoverOpenSessions() 在跨午夜场景下的时长计算错误。
完善 CLAUDE.md 项目文档，添加开发路线图和已知问题清单。

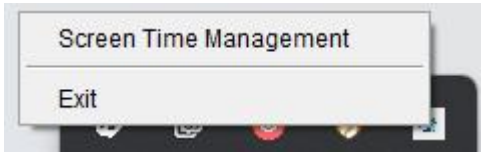
任务 14：jpackage 打包与部署
目的：实现应用程序的原生 Windows 打包，生成可独立运行的安装程序。
内容：
创建 Main.java 作为 jpackage 入口包装器，解决 jpackage 直接加载 JavaFX Application 的模块查找兼容问题。
配置 maven-assembly-plugin 生成 Fat JAR（jar-with-dependencies），编写 build.bat 脚本调用 jpackage 生成原生 Windows EXE/MSI 安装包。
验证打包后无需 JDK 环境即可在目标机器独立安装和运行。

六、测试验证截图


屏幕时间管理
监控软件管理
使用统计
统计历史记录
设置
监控软件管理
刷新进程列表
添加选中的软件到监控列表
当前运行进程
AMDRSSrcExt.exe
backgroundTaskHost.exe
cef_server.exe
clash-verge.exe
cmd.exe
cncmd.exe
Codex.exe
codex.exe
已监控的软件
移除选中
idea64 (idea64.exe)
永久显示
进程列表已刷新，共 48 个进程


屏幕时间管理
监控软件管理
使用统计
统计历史记录
设置
使用统计
刷新数据
今日使用时长
应用
使用时长
idea64
1 分钟





七、总结

本项目实现了一个功能完整的 Windows 屏幕使用时间管理工具。核心价值在于将「当前正在使用哪个应用」这一实时信息，转换为可持久化、可查询的本地时间数据，帮助用户了解自己的软件使用习惯。

主要工作包括：使用 JavaFX 构建包含 3 个功能页面的桌面界面；通过 JNA 调用 Windows 原生 API 实现前台窗口检测；设计包含监控、会话记录、日汇总 3 张表的本地 SQLite 数据库；实现完整的使用会话生命周期管理（含跨午夜拆分、异常恢复、时钟回拨保护）；通过 AWT SystemTray 实现后台运行和系统托盘最小化；以及应用程序的 jpackage 原生打包部署。

收获与心得：通过本实践深入理解了 Java 桌面应用的 MVC 分层设计，掌握了 JavaFX 的异步任务（Task）与 UI 线程的协作模式；体会到单元测试在数据库操作验证中的价值——对 UsageRecordDao 的 10 个测试覆盖了多个边界场景，帮助发现并修复了跨天会话计算的错误。

遇到的问题及解决：最棘手的是系统托盘中文字符显示为方块的问题，经过 8 种方案尝试后，最终确认根因是 JDK 21 JEP 400 将 file.encoding 默认值改为 UTF-8 后，Oracle JDK 的 fontconfig.properties 中缺少对应的中文字体回退序列，通过自定义 fontconfig.properties 文件成功解决。

建议：后续可考虑增加数据可视化的导出功能（如 PDF 报告）、支持多账户数据分离，以及将 Windows 原生 API 层抽象化以便跨平台扩展。